Sylvara — Cloud Computing & DevOps

Exam cheat sheet · Aditya Dutta · 2026-11-05

0. How to use this

Read **§1, §6 and §10** on the way in. Those three get you through the first five minutes. **§3** is the report — it is written to be paraphrased, not recited. **§7** is the viva. **§8** is the one section to read twice: it tells you which claims are load-bearing and which ones will collapse if someone pushes.

One strategic note before anything else: **you do not need to invent a cloud stack.** Sylvara already runs on a real one — Fly.io, Supabase, GitHub Actions, Docker, a Postgres-backed job queue, OTA firmware delivery, structured logging, staged environments. Every number and every service name below comes out of the project's own architecture documents. That matters, because an invented stack dies on the second follow-up question and a real one doesn't. Where something is planned rather than running, §8 says so, and gives you a sentence that is true either way.

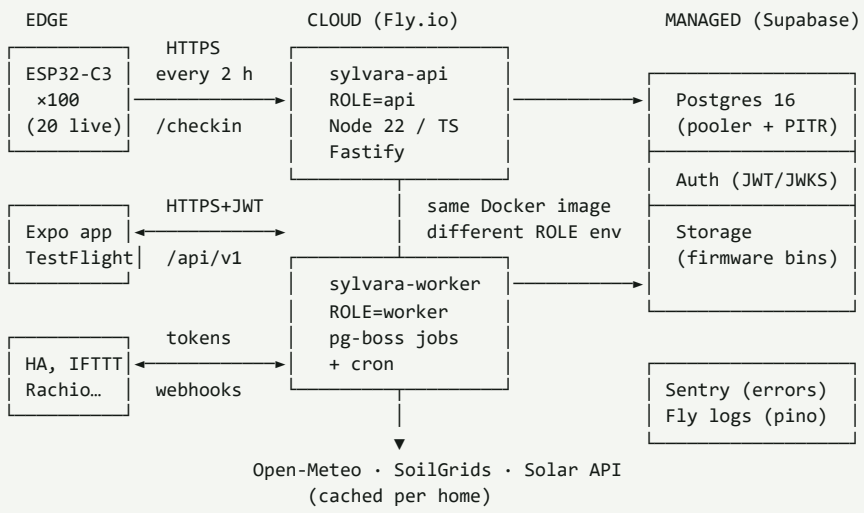
1. The 90-second answer

> *"Sylvara is a solar-powered soil sensor that buries in a plant pot and lives a year on 26 milliwatts. There are a hundred units built and about twenty deployed with beta users right now. The device is the easy part — it's asleep 99.97% of the time and wakes every two hours to post three readings over HTTPS. Everything interesting is the cloud behind it.*
>
> *That's a Node/TypeScript service containerised with Docker and deployed on Fly.io, running in two roles out of one image — an API role that answers devices and the phone app, and a worker role that runs the interpretation jobs. State lives in a managed Postgres 16 on Supabase, which also gives us Auth and object storage. Jobs and cron run on pg-boss, which is a queue inside Postgres, so there's no Redis to operate. CI is GitHub Actions: a push to main deploys staging, a git tag deploys production, and database migrations run as a release step before the new container takes traffic.*
>
> *The part I'd point at if you only gave me one thing: we ship software to hardware that is physically in another country. Firmware release is a git tag that builds inside the Espressif Docker image in CI, attaches the binary to a GitHub release, uploads it to object storage, and the devices pull it over the air against a short-lived signed URL and verify a SHA-256 before they flash it. That's a CD pipeline whose deployment target is a battery-powered device on a windowsill in Canada. The whole thing costs under forty dollars a month."*

Stop there. That paragraph contains about nine follow-up questions you already have answers for.

2. The system, in one picture

Draw this. Examiners reward a diagram, and this one you can reproduce from memory.



The two boxes in the middle are **the same container image**. The `ROLE` environment variable decides whether a machine serves HTTP or drains the job queue. If you remember one architectural detail, remember that one — it is the cleanest 12-factor example in the whole project.

---

3. The report — written to be paraphrased

3.1 What the system is, and why it needs a cloud at all

Sylvara is an IoT product: a 22 × 35 mm solar-powered board that sits in soil and measures moisture, conductivity, temperature, humidity and light. The device is deliberately stupid. It wakes, reads its sensors, and on every third wake it opens a TLS connection, posts a small JSON batch, and goes back to deep sleep at 40–50 microamps. Its total duty cycle is about 0.03%.

That constraint is the reason the cloud architecture looks the way it does. A device that is asleep 99.97% of the time cannot be pushed to, cannot hold a connection, cannot run a model, and cannot be asked to change its behaviour on demand. Everything the product actually *does* — turning a raw capacitance number into "this plant needs water in about 41 hours" — has to happen somewhere else, and that somewhere else has to be available when the device wakes up, whatever time zone it is in.

So the design rule we adopted early was: **raw forever, derived disposable.** The server stores exactly what the device sent and never modifies it. Everything computed from it carries a `model_version` and can be thrown away and recomputed. That single rule is what makes the system cloud-shaped rather than device-shaped — it means we can improve the product for a year without touching a single piece of hardware, which is just as well, because the hardware is on another continent.

3.2 Service model — what we rent and what we run

The deployment is a deliberate hybrid across service models.

PaaS — Fly.io. We hand Fly a Docker image and a `fly.toml`; it runs the container on Firecracker microVMs, terminates TLS on our custom domain, and gives us health checks, log aggregation and rollback. We never see an operating system, patch a kernel, or configure a load balancer. We chose this over raw IaaS deliberately: two engineers cannot also be two sysadmins.

BaaS — Supabase. Postgres 16, authentication and object storage as managed services. Auth was the clearest buy-versus-build decision in the project — Apple, Google and email sign-in, with the server verifying JWTs against a JWKS endpoint, for nothing. Postgres comes with point-in-time recovery, which we would not have implemented ourselves.

SaaS — the periphery. Sentry for error tracking, GitHub for source and CI, Open-Meteo and SoilGrids for environmental context, Expo Application Services for mobile builds.

Our own code is the thin, valuable middle: the ingest path, the interpretation pipeline, the automation safety layer and the public API. That is the product. Everything else is rented.

3.3 Containers, and why not Kubernetes

The service ships as one Docker image. Locally, `docker compose up` brings up a Postgres and a device simulator so a developer can run the whole system on a laptop with no cloud account. In CI, the same image is built and the contract tests run against a Postgres service container. In production, Fly runs that image on two machines with different `ROLE` values.

We consciously did not use Kubernetes, and this is worth defending rather than apologising for. Kubernetes solves scheduling across a fleet of nodes with heterogeneous workloads and a team to operate it. We have one service, two roles, under a hundred devices checking in every two hours, and two engineers. A control plane we have to operate is a liability, not an asset, at this scale. The honest trigger for reopening that decision is multi-region or multi-service — if we end up with five services and a need to run in the EU for data residency, the argument changes. Choosing the smallest thing that works, and writing down what would make you change your mind, *is* the engineering judgement the question is testing.

The same reasoning produced the other unusual choice: **no Redis.** Our job queue is `pg-boss`, which implements queues, retries, exponential backoff, singleton jobs and cron inside the Postgres database we already run. One fewer stateful service to back up, monitor, secure and pay for.

3.4 The CI/CD pipeline

There are three separate delivery pipelines, which is unusual and is the most interesting thing to talk about.

Backend. Branch per task, PR into a protected `main`. On every push, CI runs the test matrix — a byte-for-byte contract test of the device wire protocol, golden-file tests of the interpretation models, a two-user tenancy suite generated from the OpenAPI spec, and property tests asserting that re-posting any batch and re-running any job change nothing. `gitleaks` scans every push for credentials, and Renovate plus `npm audit` watch dependencies. A merge to `main` deploys **staging**. A git tag deploys **production**. Database migrations are plain SQL files applied by `node-pg-migrate` as a release step *before* the new image takes traffic, so the schema is never behind the code that expects it.

Firmware. This is continuous delivery where the deployment target is a battery. A git tag `vX.Y.Z` triggers a build inside the `espressif/idf:v5.5.4` container — pinning the toolchain in an image is what makes a firmware build reproducible on a machine that isn't the one it was developed on. The version string is derived from the tag rather than hardcoded. The binary is attached to the GitHub release and uploaded to a private Storage bucket. Devices discover it on their next check-in, download it over a short-lived signed URL, verify a SHA-256, flash to the inactive partition, and roll back automatically if the new image fails to confirm itself. Any rollback raises an alert to us.

Mobile. Expo with EAS Build and EAS Update. The app went from a dev client to a build on the App Store's tester track in three weeks, published under a placeholder name because the trademark is still clearing. EAS Update is worth naming in a cloud exam: it pushes JavaScript bundles over the air without going through app review, which is the mobile equivalent of a hotfix deploy.

3.5 Scalability and statelessness

The API role is stateless — no session affinity, no local disk, all configuration from environment variables, logs to stdout. That is what makes it horizontally scalable: add machines, and because each one is identical and holds nothing, requests can land anywhere. The worker role scales independently, because job throughput and HTTP throughput are different problems.

The real scaling constraint is the database, not the application, which is the normal answer and the honest one. Postgres connections are the scarce resource, so the API role goes through Supabase's connection pooler while migrations connect directly.

Two design choices do most of the scaling work before any machine is added. First, the hot path is tiny: a check-in inserts rows and returns — p99 under 200 ms, typically around 41 ms — and all interpretation is enqueued as a debounced background job, so a device never waits for analysis. Second, ingest is **idempotent**: readings are inserted

with `ON CONFLICT (device_id, ts) DO NOTHING`, so a device that doesn't hear our response and retries costs us nothing and corrupts nothing. Idempotency at the edge is what makes an unreliable network a non-event.

Load is verified rather than assumed: a fleet simulator runs a thousand virtual devices at accelerated cadence against staging.

3.6 Reliability, observability and operations

Structured JSON logs from `pino`, every line tagged with a request ID, a home ID and a device ID, shipped to Fly's log aggregation. Sentry carries the same tags, so an exception can be traced back to the exact device that caused it. Two health endpoints: `/healthz` says the process is alive, `/readyz` says the database and the queue are reachable — Fly polls them and won't route traffic to a machine that fails.

Metrics are deliberately fleet-shaped rather than request-shaped. A daily rollup job writes a `fleet_metrics` table, and the admin dashboard shows devices by firmware version, 24-hour check-in success rate, OTA state, battery distribution and a last-seen histogram. For this product, "how many devices failed to check in today" matters more than p50 latency.

Alerts are tied to the same idea: check-in success below 90% over six hours, any OTA rollback, queue depth, connector error spikes.

Backups are Supabase point-in-time recovery plus a weekly logical dump to object storage, and the restore is rehearsed before beta, because a backup you have never restored is a hypothesis.

3.7 Security

Secrets never enter the repository — this is enforced, not merely intended, by `gitleaks` on every push, after an SSID leaked once while the repo was public. Runtime secrets live in Fly's encrypted secret store and are injected as environment variables.

Devices authenticate with a serial derived from the chip's eFuse MAC plus a 43-character key, stored only as a SHA-256 hash. Authentication failures return a uniform error regardless of cause, which prevents an attacker from enumerating valid serials.

Users authenticate through Supabase Auth; the server verifies the JWT and then applies its own authorisation — every row hangs off a `home_id` and every query goes through a single `authorize(user, home, role)` check. We chose application-level authorisation over database row-level security so the rules live in one tested codebase, and we test it with a two-user suite that asserts cross-tenant access fails on every endpoint.

Third-party irrigation tokens are encrypted at rest with a libsodium secretbox under a server key, never logged, and rotatable by a job. And because this system can turn on water, every water-dispensing action passes a server-side policy module no integration can bypass: explicit per-zone opt-in, daily caps, minimum intervals, a rain-forecast skip, a refusal to act on stale data, stop-first ordering, post-run verification and a 90-day audit trail. A wrong insight is a bad day; a wrong valve is a flood.

Data rights are handled for PIPEDA and GDPR with export and delete endpoints that cascade and unclaim devices.

3.8 Cost

Under forty dollars a month at pilot scale: roughly ten for Fly, twenty-five for Supabase Pro, two for the domain, Sentry on the free tier, weather and map APIs within free quotas. That holds well past a thousand devices. The architecture choices that produced that number — no Redis, no Kubernetes, no managed queue, one image serving two roles — were cost decisions as much as complexity decisions, and in a two-person company those are the same decision.

---

4. Concept → evidence table

This is the mark-scoring table. If a question names a concept, name the evidence.

CONCEPT	WHERE IT APPEARS IN SYLVARA
IaaS	Not used directly — Fly's Firecracker microVMs are the closest layer, and we deliberately don't manage them
PaaS	Fly.io: push an image, get TLS, health checks, log aggregation, rollback
BaaS / SaaS	Supabase (Postgres, Auth, Storage); Sentry; GitHub Actions; Expo EAS; Open-Meteo
Serverless / FaaS	Scheduled Fly machines and pg-boss cron. Supabase Edge Functions (Deno) were specified in July and rejected — see §7 Q14
Containerisation	One Docker image, two roles. `docker compose` for local dev. `espressif/idf` image for reproducible firmware builds
Orchestration	Fly Machines. Kubernetes consciously rejected at this scale (§3.3)
Horizontal scaling	Stateless API role; worker scales independently; DB connection pooler as the real limit
Elasticity	Load is predictable and periodic (2-hour cadence), so we size rather than autoscale — an honest answer
Multi-tenancy	Shared database, shared schema, `home_id` tenancy with an application-level authorize() gate
Statelessness / 12-factor	Config in env, no local disk, logs to stdout, one codebase → many deploys, dev/prod parity via compose
Edge computing	The device aggregates on-board and uploads every third wake — processing at the edge to save bandwidth and power
Managed database	Supabase Postgres 16 with PITR; pooler for the app, direct connection for migrations
Object storage	Supabase Storage, private bucket, short-lived signed URLs, SHA-256 verified by the client
Message queue / async	pg-boss on Postgres — retries, backoff, singleton jobs, cron, no Redis
Event-driven architecture	A single `events` outbox table written in the same transaction as the state change; webhooks, push, rules and MQTT all consume from it. *This is the transactional outbox pattern — name it.*
Microservices vs monolith	Modular monolith, split by role not by service. Justified by team size; ML arrives later as a genuinely separate Python worker
CI	GitHub Actions: contract, golden-file, property, tenancy and safety tests; gitleaks; Renovate; npm audit
CD	Push to `main` → staging; git tag → prod; migrations as a pre-traffic release step
IaC	`fly.toml`, the Dockerfile and versioned SQL migrations are the declarative artifacts. No Terraform — see §8
Environments	local (compose + simulator) → staging → prod
Blue-green / canary / rollback	Rollback by redeploying the previous image; firmware OTA rolls back automatically on failed confirmation
Monitoring & logging	pino structured logs, Sentry, `/healthz` + `/readyz`, `fleet_metrics` daily rollup, threshold alerts
Backup & DR	PITR plus weekly logical dump to object storage; restore rehearsed before beta
Secrets management	Fly secrets; gitleaks in CI; connector tokens encrypted at rest with libsodium
API gateway / contracts	Fastify + Zod schemas generate the OpenAPI spec, which generates the mobile client — one definition of every payload
Load testing	1,000-device fleet simulator against staging
Cost optimisation	< $40/month; every "we didn't use X" was partly a bill

5. The three pipelines, as diagrams

Backend

```
branch → PR → CI (contract · golden · property · tenancy · safety · gitleaks)
  → merge to main ──→ build image ──→ migrate ──→ deploy STAGING
  → git tag v*     ──→ same image  ──→ migrate ──→ deploy PROD
```

Firmware

```
git tag vX.Y.Z → CI builds in espressif/idf:v5.5.4 container
  → binary attached to GitHub release
  → uploaded to Storage (firmware_releases)
  → device sees it on next 2 h check-in
  → downloads via signed URL → verifies SHA-256 → flashes inactive partition
  → confirms, or auto-rolls back and alerts us
```

Mobile

```
expo dev client → EAS Build → TestFlight (placeholder name, trademark pending)
                → EAS Update pushes JS bundles OTA, no app review
                → Maestro E2E flows run against staging
```

---

6. Numbers to memorise

NUMBER	WHAT IT IS
100 / 20	Units built / deployed with beta users
2 hours	Check-in cadence (`wake_interval_s = 7200`, `upload_every_n = 3`)
0.03%	Device duty cycle
40–50 µA	Measured deep-sleep current
26.3 mW	Real solar harvest
~450 mAh	Actual cell capacity (the label claims 1500 — physically impossible for that pouch)
22 × 35 mm	Board size
41 ms / 200 ms	Observed check-in time / p99 budget
48 tables, 12 migrations	Schema size
38 endpoints	Public + device API
24 models	Versioned interpretation functions
1,000	Virtual devices in the load simulator
0	Firmware changes required during a full month of server work — the wire contract is frozen
< $40/month	Total cloud cost at pilot scale
Node 22 · Postgres 16 · ESP-IDF 5.5.4	Version numbers worth having exact
2	Engineers

The last number is the one that makes every architectural decision defensible. Use it.

---

7. Viva Q&A

Tier 1 — you will be asked these

Q1. What's your project and where is the cloud in it? See §1. Lead with the device being asleep 99.97% of the time, because that single fact explains every subsequent decision.

Q2. Which service model are you using — IaaS, PaaS or SaaS? "All three, at different layers, and on purpose." Fly.io is PaaS — we hand it a container and never touch an OS. Supabase is BaaS/SaaS — managed Postgres, Auth and Storage. Sentry, GitHub and Expo EAS are SaaS. We use essentially no raw IaaS, because with two engineers, time spent being a sysadmin is time not spent on the product.

Q3. Public, private or hybrid cloud? Entirely public cloud, multi-provider. Not hybrid in the enterprise sense — we own no infrastructure. If asked whether there's an on-prem component: the sensor fleet is the closest thing, and it's better described as the edge tier than as private cloud.

Q4. Why containers? What does Docker actually buy you here? Three things. Dev/prod parity — the image CI tests is the image that runs. One artifact, two roles, selected by an environment variable, so there is no second build to keep in sync. And reproducible firmware builds: pinning the Espressif toolchain in `espressif/idf:v5.5.4` means a firmware binary is a function of the tag, not of whose laptop built it.

Q5. Walk me through your CI/CD pipeline. §3.4 and §5. Emphasise the branch-protection → test-matrix → staging-on-merge → prod-on-tag flow, and that **migrations run as a release step before the new image takes traffic.** If you say nothing else about CD, say that — it's the detail that shows you've actually deployed something with a database behind it.

Q6. How do you handle secrets? Fly's encrypted secret store at runtime, `.env` gitignored locally, `gitleaks` on every push in CI. Third-party tokens are additionally encrypted at rest in the database with a libsodium secretbox under a server key, so a database dump alone doesn't yield working credentials. The policy exists because the repo was public once and an SSID leaked — that's a real incident with a real control attached, which is a better answer than policy for its own sake.

Q7. How does your system scale? §3.5. Stateless API role scales horizontally; worker scales independently; the database connection count is the real ceiling, handled by a pooler. Then the two structural points: the hot path defers all work to a queue, and ingest is idempotent, so retries are free. Finish with the fleet simulator — you've tested to a thousand devices.

Q8. What happens if the server is down when a device wakes up? The device backs off and retries on its next cycle. Readings are timestamped on the device as `now - age_s`, so nothing loses its true time while queued, and `ON CONFLICT DO NOTHING` means a retry that partially succeeded before can be replayed safely. Availability of the cloud is not on the critical path for data integrity — which is exactly the property you want when your clients are on solar power.

Q9. How do you monitor it? §3.6. Structured logs with correlation IDs → Sentry with matching tags → `/healthz` and `/readyz` wired to platform health checks → a daily `fleet_metrics` rollup → threshold alerts. The line worth saying: *"we monitor fleet health, not request latency, because for this product the question is how many devices missed a check-in, not what the p50 was."*

Q10. Backups? Point-in-time recovery from the managed Postgres, plus a weekly logical dump to object storage so we're not solely dependent on one vendor's console. And the restore gets rehearsed — an untested backup is a hypothesis, not a backup.

Tier 2 — likely follow-ups

Q11. Monolith or microservices? Justify. Modular monolith, split by *role* rather than by *service*. The interpretation pipeline is the product, and it wants one tested codebase with shared types, not a network hop between every stage. We get the operational benefit people usually want from microservices — independent scaling of the API and the worker — out of one image and an env var. The honest reopening condition: the ML layer arrives as a genuinely separate Python worker, because it has a different runtime and a different release cadence. That's a real service boundary. "We put the boundary where the runtime differs, not where the org chart does."

Q12. Why not Kubernetes? §3.3. Do not be defensive. "Kubernetes solves scheduling heterogeneous workloads across a node fleet, with a team to operate the control plane. We have one service, two roles and two engineers. The right answer at this scale is the smallest thing that works, and I can tell you exactly what would change my mind — multi-region, or more than a handful of services." Examiners reward the reopening condition more than the decision.

Q13. Why Postgres as a job queue instead of Redis or RabbitMQ? pg-boss gives retries, exponential backoff, singleton jobs and cron. Everything we need is there. A separate broker is another stateful service to back up, monitor, secure and pay for, and it introduces the hardest bug class in distributed systems — a job committed to the queue while its database transaction rolls back. With the queue *in* the database, enqueue and state change are one transaction. That's the transactional outbox pattern, and it's the reason the choice is principled rather than just cheap.

Q14. Did you consider serverless? Why didn't you go all-in on it? Yes — there's a complete specification from July for a pure Supabase Edge Functions (Deno) deployment, with SQL, row-level security and the edge functions written out. We chose not to deploy it. The reasoning: the interpretation pipeline is 24 versioned functions with golden-file tests and shared types, and that wants one tested codebase we own, not logic spread across a console in SQL policies and edge functions. The wire contract is identical either way, so the firmware doesn't care — which is itself the point. We do use serverless-shaped things where they fit: cron jobs and scheduled machines.

Q15. Where's your Infrastructure as Code? Be straight here. "`fly.toml`, the Dockerfile and versioned SQL migration files are our declarative infrastructure — the environment is reproducible from the repository. We don't use Terraform, because we have two managed providers and about six resources between them; Terraform earns its keep when you have enough cloud resources that drift becomes real. If we added a third provider or a second region, I'd introduce it." That is a better answer than pretending.

Q16. How do you do zero-downtime deploys? Fly starts the new machine, polls `/readyz`, and only then shifts traffic; the old one drains. Migrations run before the new image takes traffic and are written additively, so the old and new code can both run against the schema during the overlap — expand first, contract later. Rollback is redeploying the previous image, which is an immutable artifact, so it's always available.

Q17. How do you update devices in the field — and what if an update bricks one? §5. Tag → container build → signed artifact → device pulls on its next wake → SHA-256 verify → flash to the *inactive* partition → confirm. If the new image doesn't confirm itself, the bootloader falls back to the previous partition and we get an alert. The reason this is non-negotiable: the only hardware we can physically touch is on another continent, and twenty un-updatable units would be twenty flights.

Q18. What's your testing strategy in CI? Five layers. Contract — the device wire protocol byte-for-byte against a Postgres service container. Golden files — synthetic sensor series producing known interpretation outputs, so any change to expectations is a reviewed diff. Property — re-posting any batch or re-running any job changes nothing. Integration — two users, cross-tenant access must fail on every endpoint, generated from the OpenAPI spec. Safety — every rule that can block a water action has a test proving it blocks it.

Q19. How do you handle multi-tenancy? Shared database, shared schema. Every user-reachable row hangs off a `home_id`, and every query goes through one `authorize(user, home, role)` function. We chose application-level authorisation over Postgres row-level security so the logic is in one place, in the language the rest of the service is written in, and testable in CI. The trade-off is honest: RLS would be enforced even if application code has a bug, so we compensate with a generated test suite that tries cross-tenant access on every endpoint.

Q20. How does the mobile app fit into the cloud architecture? It's deliberately thin — every screen is a view of one server endpoint, and the client is generated from the same OpenAPI spec the server validates against, so a contract change breaks the build rather than production. It authenticates through Supabase Auth and never touches the database directly. It's on the App Store tester track under a placeholder name while the trademark clears. The one non-thin part is Bluetooth provisioning, because that talks to the device directly rather than through the cloud.

Q21. What's your disaster-recovery story? PITR covers data loss. Immutable images cover a bad deploy. OTA rollback covers bad firmware. The weekly off-platform dump covers the provider itself. The gap I'd name unprompted: we're single-region, so a regional outage is downtime — acceptable for a product whose clients retry in two hours, and not acceptable at a different scale. Naming your own gap before the examiner does is worth marks.

Q22. How do you manage configuration across environments? Environment variables, nothing else — local, staging and prod run the same image and differ only in injected configuration. `ROLE` selects api or worker. Device-side config is server-driven too: the check-in response carries the wake interval and upload cadence, both clamped to valid ranges on *both* sides, so a bad server value can't put a device to sleep for a month.

Tier 3 — curveballs

Q23. What's the hardest cloud problem you hit? Idempotency at the edge. A device that transmits, doesn't hear the response, and retries, must not create duplicates — on a cellular-grade link to a solar-powered device, that is not an edge case, it's Tuesday. The fix is a natural key of `(device_id, ts)` with the timestamp derived on the device from the reading's age, plus `ON CONFLICT DO NOTHING`. Retries became free, and a whole class of data-quality bugs disappeared.

Q24. CAP theorem — where does your system sit? Firmly CP at the database, because it's a single Postgres: under a partition we lose availability, not consistency. But the *system* tolerates that because the clients are asynchronous — a device that can't reach us retries in two hours and loses nothing. The interesting observation is that the device fleet behaves like an eventually-consistent AP layer sitting in front of a CP core, which is why the CP choice costs us almost nothing in practice.

Q25. Vendor lock-in? Real and bounded. Postgres is portable — it's SQL and plain migration files. The container runs anywhere Docker runs; Fly is replaceable by Railway or Render in a day. The genuine lock-in is Supabase Auth and Storage, and that was a deliberate trade: auth is a thing you should not build, and the migration cost if we ever leave is a known, finite project rather than an unknown one.

Q26. How would this change at 100,000 devices? Honestly and specifically: read replicas for the dashboard queries; the readings table partitioned by time with a retention policy pushing cold data to object storage; the worker split by job type so a slow interpretation run can't starve the OTA jobs; and multi-region, at which point the "no Kubernetes" decision genuinely reopens. The cadence helps — a hundred thousand devices on a two-hour cycle is about fourteen writes a second if they're jittered properly, and *jittered properly* is doing a lot of work in that sentence. Thundering-herd avoidance is the first thing I'd implement.

Q27. What would you do differently? Hosting and CI should have come before features. We ran the backend on a laptop behind a Windows firewall rule, with the device's server address compiled into the firmware — so when the hardware left the country it was unreachable, and the fix needed a firmware release we couldn't deploy. The rule that came out of it is written down verbatim: *never bake a LAN IP into firmware again*. Everything about the current architecture — OTA first, a Bluetooth-writable server URL, a real hostname — is a reaction to that one mistake.

Q28. How do you do capacity planning / what's your cost model? §3.8. Fixed-cost architecture at pilot scale, deliberately — a flat $40/month with no per-request pricing means no surprise bill and no incentive to under-instrument. The variable costs that will appear first are map tiles and weather API calls, which is why context lookups are cached per home rather than per device: all the devices in one home share a weather query.

Q29. What is DevOps, actually? Don't recite CALMS. Answer from the project: "It's the practice of making the path from a commit to a running system short, automated and reversible — and in our case the system includes a hundred battery-powered devices we can't reach. The culture part is real but it's downstream of the tooling: once a deploy is a tag and a rollback is a redeploy, nobody is afraid to ship, and the arguments stop being about who broke production."

Q30. Why did you build it this way instead of using AWS? Because the right question isn't which cloud, it's how much cloud two people can operate while also building hardware. The equivalent AWS stack — ECS or EKS, RDS, Cognito, S3, CloudWatch, Secrets Manager, CodePipeline — is more capable and strictly more work, and the IAM surface alone would be a week. We chose the smallest set of managed services that covers the requirement. Every piece has an AWS equivalent I can name, which is the point: the architecture is portable, the vendor isn't the architecture.

Q31. Have you used Jenkins / Ansible / Prometheus / Grafana / Terraform? Say what's true and map it: "We use GitHub Actions rather than Jenkins — same concept, hosted runners instead of a server we maintain. Configuration management is handled by containers rather than Ansible, because immutable images make convergence a non-problem. Metrics are Sentry plus our own rollup table rather than Prometheus and Grafana; the honest reason is that fleet metrics mattered more than request metrics at this size, and OpenTelemetry is on the list." Naming the equivalence shows you understand the category, not just the tool.

Q32. What does "immutable infrastructure" mean here? A deployed version is an image digest, not a machine someone configured. Nothing is patched in place; to change the system you build a new image and replace the old one. It's why rollback is trivial and why "works on my machine" doesn't come up — the artifact CI tested is the artifact that runs.

Q33. What's the single best decision you made? Freezing the device wire contract. A full month of server work — 48 tables, 38 endpoints, 24 models — required *zero* firmware changes. That's the whole argument for stable interfaces in one number, and it's why the system could evolve while the hardware was physically unreachable.

---

8. Thin ice — read this twice

Everything in §1–§7 is drawn from the project's real architecture and operations documents. A few things need care.

Solid. Lean on these. The device fleet and its numbers. Fly.io + Supabase + Sentry as the hosting choice and the reasoning behind it. The Docker image with two roles. The staging-on-merge / prod-on-tag flow and migrations as a release step. The firmware OTA pipeline through a pinned toolchain container. pg-boss and the no-Redis argument. The transactional outbox. Idempotent ingest. gitleaks and the secret-leak incident. The tenancy model. The water-safety policy. The schema and endpoint counts. The cost figure.

True but in progress — phrase in the present-continuous, not the past. Several integrations (Rachio, Netro, LinkTap, Gardena) are designed and partly built rather than live; say "the connector framework is built and we're bringing partners on" rather than "we integrate with Rachio." The MQTT / Home Assistant bridge is built but optional. The restore rehearsal is scheduled, not necessarily done — "we rehearse restores before beta" is safe; "we restored last month" is not.

Do not claim these. Kubernetes. Terraform or any IaC tool. Jenkins. Prometheus or Grafana. Multi-region or multi-cloud failover. Autoscaling policies. OpenTelemetry tracing. Any machine learning in production — **v1 ships zero ML by explicit decision**, and the ML work you have is a disclosed synthetic-data study, so if ML comes up, describe it as the *planned* v2 Python worker and the architecture that's ready for it. Each of these has a prepared answer in §7 (Q12, Q15, Q31) that scores better than a bluff, because the follow-up question to "yes we use Kubernetes" is "show me your manifests."

On authorship. You are the co-founder and technical lead; the backend and app were built with your co-founder, and the architecture decisions in this document are recorded as joint and founder decisions. The frictionless and accurate register is first-person plural for the build and first-person singular for the decisions and the hardware: *"we built", "I decided", "the constraint I was designing against."* It's natural, it's what founders actually sound like, and it survives the question "who wrote this part?" in a way that "I built all of it" does not. If someone asks directly whether you worked alone: *"Two of us, remote — I own hardware, firmware and architecture, my co-founder and I built the backend and app together."* That costs you nothing and removes the only question in the room you can't answer.

On the app's name. It's on TestFlight under a placeholder because the trademark is still clearing. That's genuinely true and it's a *good* line — it makes you sound like someone shipping a real product rather than a student project.

---

9. Vocabulary to drop, naturally

Use these when they fit; a correctly-used term is worth a paragraph of description.

Transactional outbox · idempotency key · natural key · immutable infrastructure · dev/prod parity · twelve-factor · expand-and-contract migration · connection pooling · backpressure · debouncing · thundering herd · jitter · exponential backoff · blue-green · canary rollout · signed URL · JWKS · bearer token scope · HMAC-signed webhook · replay window · multi-tenancy (shared schema) · least privilege · secret rotation · PITR · RPO / RTO · graceful drain · readiness vs liveness probe · structured logging · correlation ID · golden-file test · contract test · service boundary · vendor lock-in · managed service · control plane

Three to use without being asked, because they mark you out: **transactional outbox**, **expand-and-contract migration**, **readiness versus liveness**.

---

10. If you only memorise five things

1. **The device is asleep 99.97% of the time.** Every architectural decision falls out of that, and saying it first makes you sound like you designed a system rather than assembled one.
2. **One Docker image, two roles, chosen by a `ROLE` environment variable.** Your cleanest containerisation and twelve-factor example in a single sentence.
3. **Push to `main` deploys staging; a git tag deploys production; migrations run before the new image takes traffic.** That last clause is the one that sounds like experience.
4. **The queue is inside Postgres, so enqueueing a job and changing state are one transaction** — the transactional outbox, which is also why there's no Redis.
5. **Two engineers, under $40 a month, and the hardware is on another continent.** This is the frame that turns every "why didn't you use X" into a strength instead of a gap.

And the closing line, if you get to choose one: *"A month of server work, zero firmware changes. Freeze the contract, then build behind it."*